Frontend Build Configuration

<details>
<summary>Relevant source files</summary>

- `Frontend/STUDER/angular.json`
- `Frontend/STUDER/package-lock.json`
- `Frontend/STUDER/package.json`
- `Frontend/STUDER/src/index.html`
- `Frontend/STUDER/tsconfig.app.json`
- `Frontend/STUDER/tsconfig.json`
- `Frontend/STUDER/tsconfig.spec.json`
</details>

This section details the build infrastructure and configuration for the STUDER Angular 19 application. It covers the transition to the modern Angular application builder (esbuild/Vite), TypeScript compilation settings, and the automated testing environment.

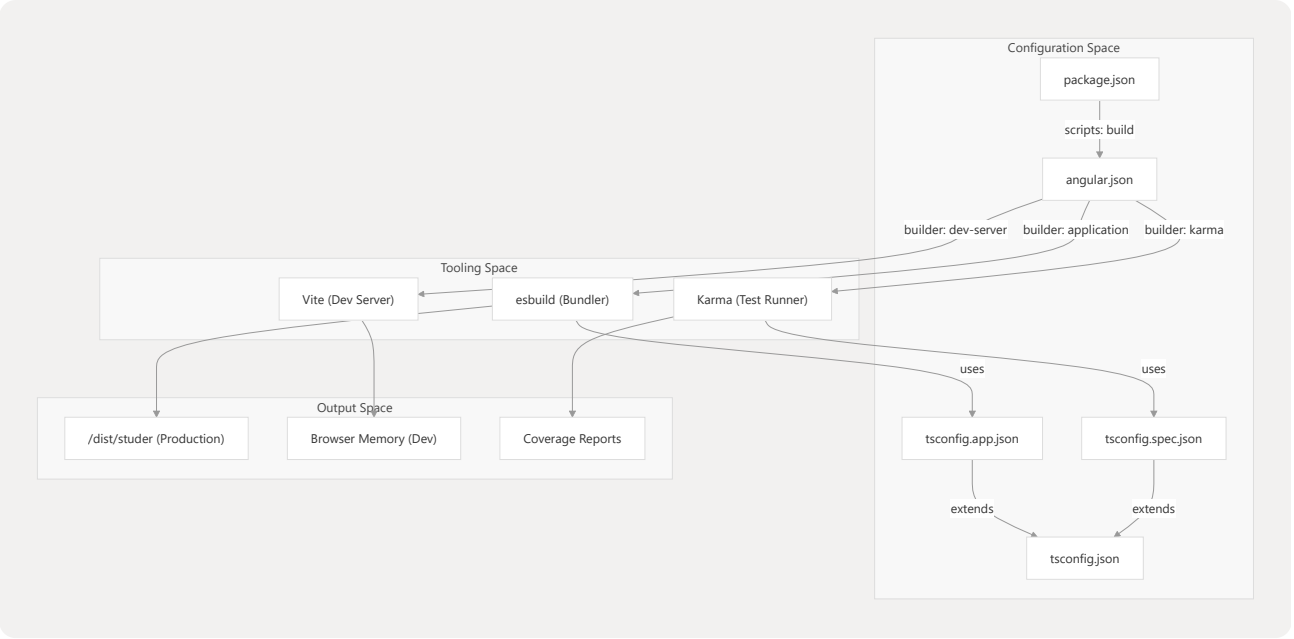
Build System Overview

STUDER utilizes the Angular CLI version 19 toolchain. The build process has been migrated to the modern `@angular-devkit/build-angular:application` builder, which leverages **esbuild** for bundling and **Vite** for the development server, providing significantly faster build times compared to the legacy Webpack-based pipeline `Frontend/STUDER/angular.json 14`

Build Lifecycle Diagram

The following diagram illustrates the relationship between configuration files and the build output.

Figure 1: Configuration to Build Output Mapping



Sources: `Frontend/STUDER/package.json 4-10` `Frontend/STUDER/angular.json 13-98` `Frontend/STUDER/tsconfig.app.json 4` `Frontend/STUDER/tsconfig.spec.json 4`

Workspace Configuration (`angular.json`)

The **`angular.json`** file defines the architect targets for the **`STUDER`** project

`Frontend/STUDER/angular.json 6`

Key Build Targets

Target	Builder	Description
`build`	**`@angular-devkit/build-angular:application`**	Compiles the app using esbuild. Outputs to **`dist/studer`** `Frontend/STUDER/angular.json 14-16`
`serve`	**`@angular-devkit/build-angular:dev-server`**	Runs the Vite-based development server `Frontend/STUDER/angular.json 65`
`test`	**`@angular-devkit/build-angular:karma`**	Executes unit tests using Karma and Jasmine `Frontend/STUDER/angular.json 80`

Production vs Development Configurations

The `build` target includes two primary configurations:

1. **Production**: Enables `outputHashing: "all"` for cache busting and defines strict `budgets` (e.g., 1MB maximum error for the initial bundle) `Frontend/STUDER/angular.json 41-55`
2. **Development**: Disables optimization, enables source maps, and prevents license extraction to speed up local iteration `Frontend/STUDER/angular.json 56-60`

Sources: `Frontend/STUDER/angular.json 13-80`

TypeScript Configuration

The project uses a hierarchical TypeScript configuration to separate concerns between application code and test code.

Base Settings (`tsconfig.json`)

The root configuration enforces strict type checking and modern ECMAScript standards:

- **Target/Module**: `ES2022` `Frontend/STUDER/tsconfig.json 18-19`
- **Strictness**: `strict: true`, `noImplicitReturns: true`, and `noFallthroughCasesInSwitch: true` `Frontend/STUDER/tsconfig.json 7-11`
- **Angular Specifics**: `strictTemplates: true` and `strictInjectionParameters: true` are enabled to catch errors in HTML templates during build time `Frontend/STUDER/tsconfig.json 23-25`

Application and Test Variants

- `tsconfig.app.json`: Extends the base config and explicitly includes `src/main.ts` as the entry point `Frontend/STUDER/tsconfig.app.json 4-10`
- `tsconfig.spec.json`: Extends the base config, adds `jasmine` types, and includes all `**/*.spec.ts` files for testing `Frontend/STUDER/tsconfig.spec.json 4-12`

Sources: `Frontend/STUDER/tsconfig.json 1-27` `Frontend/STUDER/tsconfig.app.json 1-15` `Frontend/STUDER/tsconfig.spec.json 1-15`

Dependency Management and Scripts

The `package.json` file manages the runtime and development dependencies, as well as the CLI shortcuts.

NPM Scripts

Common development tasks are mapped to the following scripts:

- `npm start`: Alias for `ng serve`. Launches the dev server `Frontend/STUDER/package.json 6`
- `npm run build`: Alias for `ng build`. Generates production artifacts `Frontend/STUDER/package.json 7`
- `npm test`: Alias for `ng test`. Runs the test suite `Frontend/STUDER/package.json 9`

Core Toolchain Components

The application relies on several key packages for its UI and functionality:

- **Framework**: Angular 19.2 `Frontend/STUDER/package.json 13-19`
- **UI Library**: PrimeNG 19.1.4 and PrimeIcons `Frontend/STUDER/package.json 23-24`
- **Styling**: FontAwesome 7.2 `Frontend/STUDER/package.json 20`
- **I18n**: `@ngx-translate/core` `Frontend/STUDER/package.json 21`

Sources: `Frontend/STUDER/package.json 1-42` `Frontend/STUDER/package-lock.json 10-39`

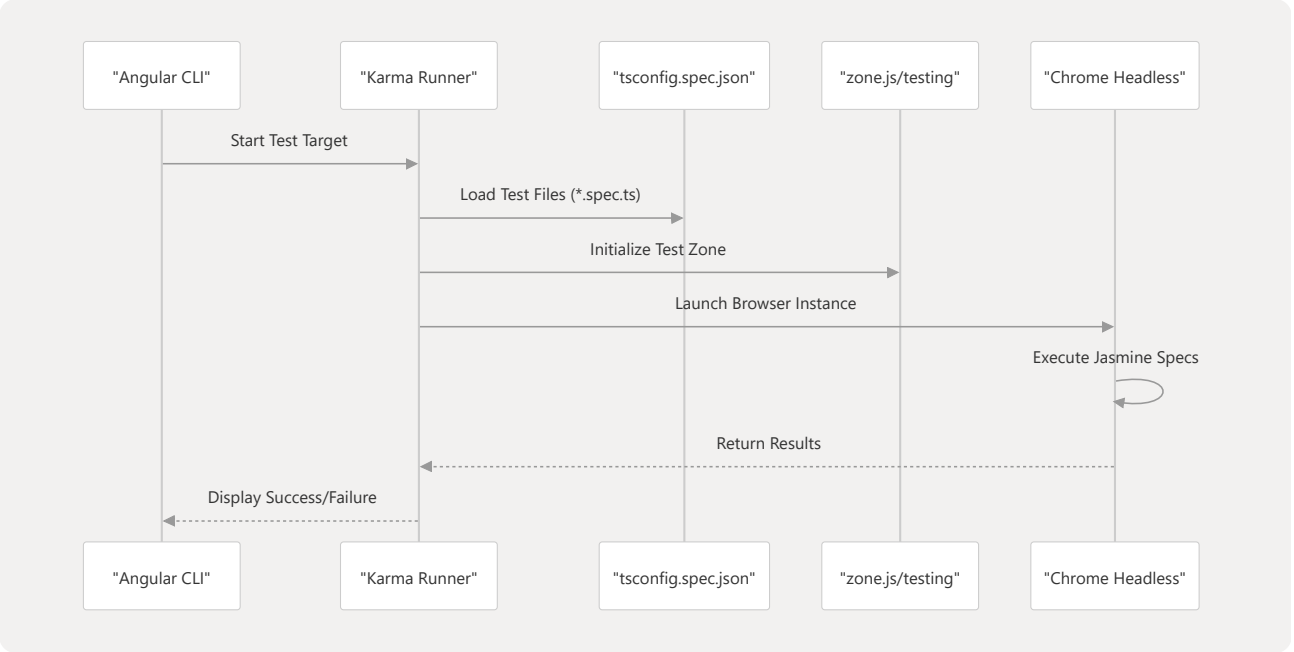
Testing Infrastructure

STUDER uses **Karma** as the test runner and **Jasmine** as the BDD (Behavior-Driven Development) framework.

Test Execution Flow

When `ng test` is executed, the following flow occurs:

Figure 2: Test Environment Initialization



Configuration Details

- **Polyfills**: The test environment requires `zone.js/testing` to handle asynchronous operations in tests `Frontend/STUDER/angular.json 82-85`
- **Reporters**: The project is configured with `karma-coverage` for generating code coverage reports and `karma-jasmine-html-reporter` for visual feedback `Frontend/STUDER/package-lock.json 35-39`

Sources: `Frontend/STUDER/angular.json 79-98` `Frontend/STUDER/package.json 33-39` `Frontend/STUDER/tsconfig.spec.json 1-15`